



Relatório sobre Projeto envolvendo coleta de dados via ESP32 e MQTT



Coleta de dados e Telemetria

Relatório Técnico apresentado à Unidade Curricular de Coleta de Dados em Automação como requisito para avaliação.

Docente: Lucas Sousa dos Santos
Jaraguá do Sul - 2026



Sumário

Sumário.....	3
1. INTRODUÇÃO.....	4
2. DESENVOLVIMENTO.....	5
2.1. Arquitetura de dados.....	5
2.2 Organização e definição de Hardware.....	5
2.2.1. Microcontrolador: ESP32-DevKit-V1.....	6
2.2.2. Sensor Temperatura: DHT22.....	6
2.2.3. Simulador Vibração: Potenciômetro 1.....	6
2.2.4. Simulador Corrente: Potenciômetro 2.....	6
2.2.5. Display Local: LCD 16x2 I2C.....	6
2.2.6. Indicador de Status: LED Verde.....	6
2.3. Códigos utilizados.....	7
2.3.1. Código em C++.....	7
2.3.2. Código em Java.....	9
2.4. Lógica e Tratamento de Dados.....	10
2.5 Testes com o Projeto.....	11
Problemas enfrentados.....	11
Passo a Passo de Utilização.....	12
Resultado dos Testes com Web Client.....	12
Atualização dos Testes.....	13
Resultado dos Testes Padrões.....	14
2.6. Gestão de Tempo e Planejamento.....	15
3. CONCLUSÃO.....	16
Referências Bibliográficas.....	17



1. INTRODUÇÃO

Este relatório documenta o desenvolvimento de um cbersistema dedicado à coleta de dados e monitoramento de motores elétricos em ambientes industriais. O projeto integra uma solução de hardware baseada no microcontrolador ESP32 e um sistema backend desenvolvido em Java. O objetivo principal é realizar a leitura contínua de parâmetros críticos, como temperatura, vibração e corrente elétrica, bem como transmiti-los em tempo real para um servidor de monitoramento central, garantindo a segurança e a previsibilidade operacional na automação 4.0.



2. DESENVOLVIMENTO

2.1. Arquitetura de dados

A arquitetura do sistema foi projetada respeitando os paradigmas adequados para cada camada tecnológica:

- Firmware (ESP32): Utilizou-se o paradigma de Programação Estruturada em C++ para otimizar o uso de memória e garantir a leitura em tempo real dos sensores, já que é a linguagem padrão para microcontroladores.
- Backend (Github Codespace): Aplicou-se a Programação Orientada a Objetos em Java, facilitando a manutenção e a escalabilidade da recepção dos dados.

No caso da comunicação de dados, a integração bidirecional foi estabelecida através do protocolo MQTT (Message Queuing Telemetry Transport), ideal para ambientes com restrição de banda e dispositivos IoT. Incluiu essas características:

- Broker Público: `broker.hivemq.com` (Porta 1883).
- Tópico de Telemetria: `senai/henrique/motor/dados`
- Formato do Payload: String simples delimitada por vírgulas (Ex: "25.5, 10, 15"), otimizando o tráfego de rede em comparação a estruturas mais pesadas como JSON.
- Segurança: Para evitar conflitos de conexão no broker público, o identificador do cliente (Client ID) no Java foi gerado dinamicamente utilizando a função `System.currentTimeMillis()`, garantindo uma sessão exclusiva a cada execução.

2.2 Organização e definição de Hardware

Aqui está uma lista dos componentes que foram utilizados durante a realização do projeto, bem como os pinos a qual foram inseridos e suas funções:

- Microcontrolador: ESP32-DevKit-V1;
- Sensor Temperatura: DHT22 (Pino GPIO 15);
- Simulador Vibração: Potenciômetro 1 (Pino GPIO 34);
- Simulador Corrente: Potenciômetro 2 (Pino GPIO 35);



- Display Local: LCD 16x2 I2C (SDA -> 21, SCL -> 22); e
- Indicador de Status: LED Verde (Pino GPIO 12 - Ligar ao iniciar envio)

2.2.1. Microcontrolador: ESP32-DevKit-V1

É o cérebro do diagrama elétrico físico, responsável por enviar os dados coletados pelo sensor DHT22 e pelos dois potenciômetros para o BackEnd Java via Broker MQTT.

2.2.2. Sensor Temperatura: DHT22

Sua função é apenas simular dados de temperatura.
Foi conectado ao pino GPIO 15 do ESP.

2.2.3. Simulador Vibração: Potenciômetro 1

Sua função é apenas simular dados de vibração do motor de forma analógica.
Foi conectado ao pino GPIO 34 do ESP.

2.2.4. Simulador Corrente: Potenciômetro 2

Sua função é apenas simular dados de corrente do motor de forma analógica.
Foi conectado ao pino GPIO 35 do ESP.

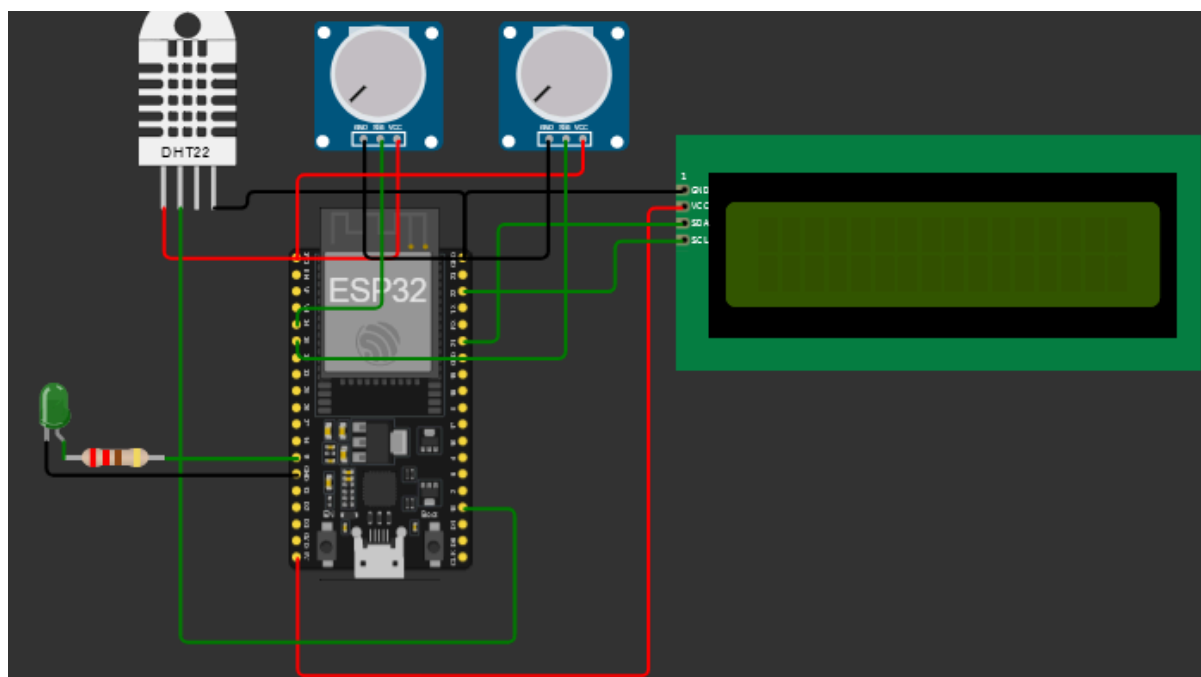
2.2.5. Display Local: LCD 16x2 I2C

Serve para mostrar os dados coletados no visor. Foi utilizado principalmente para testes, visto que o objetivo do projeto é visualizar os dados via BackEnd Java. Possui o pino SDA e SCL, sendo o primeiro conectado no GPIO 21 e o outro conectado no GPIO 22, respectivamente.

2.2.6. Indicador de Status: LED Verde

Indica quando está ocorrendo transferência de dados entre o ESP32 e o MQTT com sucesso. Foi conectado ao pino GPIO 12 do ESP, junto com um resistor de 220 Ω .

Figura 1: Diagrama Elétrico no Wokwi



Fonte: [Wokwi.com](https://wokwi.com), 2026

2.3. Códigos utilizados

Nesta seção encontra-se os códigos utilizados para execução do projeto, incluindo:

- Código em C++: utilizado no site Wokwi, para rodar no ESP32;
- Código em Java: usado no GitHub Codespace, para receber e mostrar diretamente os dados do ESP.

2.3.1. Código em C++

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

//CONFIGURAÇÕES DE REDE E MQTT
const char* ssid = "Wokwi-GUEST"; //nome da rede gratis
const char* password = ""; //sem senha
const char* mqtt_server = "broker.hivemq.com"; //broker
const char* topic_telemetria = "senai/henrique/motor/dados"; //telemetria padronizada

//DEFINIÇÃO DE PINOS
```

```
#define PIN_DHT 15
#define PIN_VIBRA 34
#define PIN_CORRENTE 35
#define PIN_LED 12

// INICIALIZAÇÃO DE OBJETOS
DHT dht(PIN_DHT, DHT22);
LiquidCrystal_I2C lcd(0x27, 16, 2);
WiFiClient espClient;
PubSubClient client(espClient);

void setup_wifi() {
  Serial.print("Conectando ao Wi-Fi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Conectado!");
}

void reconnect() { //FUNÇÃO PRA CONECTAR O CLIENTE CASO NAO ESTEJA
  while (!client.connected()) {
    Serial.print("Tentando conexão MQTT...");
    if (client.connect("ESP32_Motor_Client")) {
      Serial.println("Conectado!");
    } else {
      delay(5000);
    }
  }
}

void setup() {
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(115200);
  dht.begin();
  lcd.init();
  lcd.backlight();
  setup_wifi();
  client.setServer(mqtt_server, 1883);
}

void loop() { //FUNÇÃO PRA CONECTAR O CLIENTE CASO DESCONECTE
  if (!client.connected()) reconnect();
  client.loop();

  // 1. LEITURA DOS SENSORES
  float temp = dht.readTemperature();
  // Mapeia os valores analógicos (0-4095) para escalas didáticas
  int vibra = map(analogRead(PIN_VIBRA), 0, 4095, 0, 100);
  int corrente = map(analogRead(PIN_CORRENTE), 0, 4095, 0, 30); // Simula 0-30 Amperes

  // 2. EXIBIÇÃO LOCAL (LCD)
  lcd.setCursor(0, 0);
  lcd.print("T:"); lcd.print(temp); lcd.print(" V:"); lcd.print(vibra);
  lcd.setCursor(0, 1);
  lcd.print("Corrente: "); lcd.print(corrente); lcd.print("A");
```



```
// 3. ENVIO DOS DADOS (MQTT)
digitalWrite(PIN_LED, HIGH); // Liga ao iniciar envio [cite: 442]

// Formato: "temp,vibra,corrente" [cite: 460]
String payload = String(temp) + "," + String(vibra) + "," + String(corrente);

Serial.print("Enviando: ");
Serial.println(payload);
client.publish(topic_telemetria, payload.c_str());

delay(500); // Breve pausa para o LED ser visível
digitalWrite(PIN_LED, LOW);

delay(2000); // Aguarda 2 segundos para a próxima leitura
}
```

Fonte: Criado pelo autor com o auxílio de Inteligência Artificial (Gemini), 2026

2.3.2. Código em Java

```
/*comando para rodar no terminal:
mvn exec:java -Dexec.mainClass="senai.henrique.motor.dados.App" */

package senai.henrique.motor.dados;

import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence; // Importação adicionada

public class App {
    public static void main(String[] args) {

        String broker = "tcp://broker.hivemq.com:1883";
        String clientId = "JavaClient_Henrique_" + System.currentTimeMillis(); // inicia sessão nova
        // cada vez que roda
        String topic = "senai/henrique/motor/dados"; // Tópico padronizado conforme o documento

        // Configura o Paho MQTT para usar a memória RAM, evitando a criação daquelas pastas de
        // cache
        MemoryPersistence persistence = new MemoryPersistence();

        try {
            // Passamos o 'persistence' como terceiro parâmetro aqui
            MqttClient client = new MqttClient(broker, clientId, persistence);
            client.connect();
        }
```

```
System.out.println("Conectado ao Broker! Aguardando telemetria do motor...");

// Inscrição para receber a String simples: "temperatura,vibração,corrente"
client.subscribe(topic, (t, msg) -> {
    String payload = new String(msg.getPayload());

    // Formato de saída
    System.out.println("Dados de Telemetria Coletados com Sucesso: [" + payload + "]);
});

} catch (MqttException e) {
    System.err.println("Erro na conexão MQTT: " + e.getMessage());
}
}
```

Fonte: Criado pelo autor com o auxílio de Inteligência Artificial (Gemini), 2026

2.4. Lógica e Tratamento de Dados

No firmware (C++), os dados brutos captados pelos pinos analógicos (ADC) do ESP32 exigiram tratamento matemático para refletir grandezas físicas reais. O conversor ADC do ESP32 possui resolução de 12 bits, retornando valores entre 0 e 4095. Para converter essas leituras em dados compreensíveis para o setor de testes de motores, aplicou-se a função nativa `map()`:

- Vibração: `map(analogRead(PIN_VIBRA), 0, 4095, 0, 100);` (Simulando de 0 a 100%).
- Corrente Elétrica: `map(analogRead(PIN_CORRENTE), 0, 4095, 0, 30);` (Simulando de 0 a 30A).

Após a conversão, os valores — juntamente com a temperatura exata em *float* fornecida pela biblioteca do DHT22 — foram concatenados em uma única String separada por vírgulas, otimizando o envio através do método `client.publish()`.

2.5 Testes com o Projeto

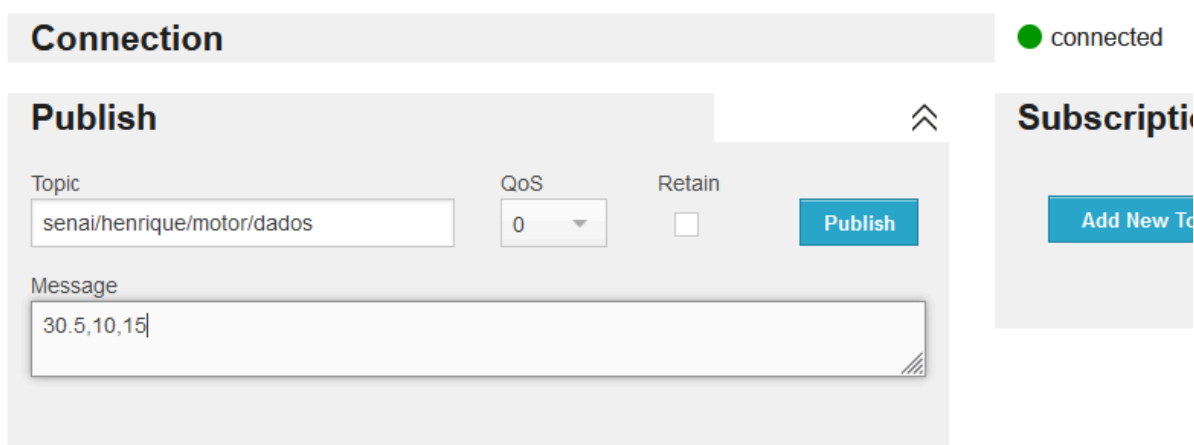
Problemas enfrentados

Durante a fase de execução dos testes de integração, a infraestrutura em nuvem do simulador de hardware, o Wokwi, apresentou superlotação e instabilidade em seus servidores, impossibilitando a inicialização do firmware desenvolvido.

Sendo assim, para garantir o cumprimento rigoroso do cronograma estabelecido e a entrega da solução no prazo, adotei um plano de contingência técnico envolvendo as seguintes situações:

1. O backend em Java foi validado de forma isolada e mantido em execução no GitHub Codespaces, visto que houve problemas com o VS Code (O Maven não estava instalado, e não há como instalar devido às limitações nos computadores do laboratório).
2. O hardware, que seria o projeto elétrico, foi simulado utilizando o cliente oficial *HiveMQ Websocket Client*, já que não foi possível utilizar o Wokwi devido às limitações de servidor mencionadas anteriormente.
3. Realizou-se a injeção manual da String de telemetria esperada (que segue esse formato: "30.5, 10, 15") diretamente no tópico *senai/henrique/motor/dados*.

Figura 2: Site do HiveMQ Web Client, onde foi feita a inserção manual de dados



The screenshot displays the HiveMQ Web Client interface. At the top, a 'Connection' status bar shows a green dot and the text 'connected'. Below this, the 'Publish' tab is active, showing a 'Topic' field with the value 'senai/henrique/motor/dados', a 'QoS' dropdown set to '0', and an unchecked 'Retain' checkbox. A blue 'Publish' button is located to the right of these fields. Below the 'Publish' section, a 'Message' text area contains the string '30.5,10,15'. To the right of the 'Publish' section, a 'Subscriptions' panel is partially visible, showing an 'Add New Topic' button.

Fonte: HiveMQ Web Client, 2026



O HiveMQ Web Client é uma ferramenta de diagnóstico e testes baseada em navegador que utiliza a tecnologia *WebSocket* para estabelecer conexão com brokers MQTT. Ele atua como um emulador de hardware, permitindo que desenvolvedores publiquem mensagens (Publish) ou assinem tópicos (Subscribe) em tempo real, sem a necessidade de dispositivos físicos ou simuladores de circuitos completos. No contexto deste projeto, ele foi essencial para isolar a camada de hardware e validar o recebimento, a decodificação e o tratamento de dados (Strings) pelo backend em Java, já que o Wokwi estava com seus servidores lotados.

Passo a Passo de Utilização

Para replicar o teste de injeção de dados simulando o ESP32, o seguinte procedimento foi adotado:

1. **Acesso:** Acessar a interface do HiveMQ Web Client através do navegador web.
2. **Conexão:** Manter os dados padrão do servidor público (*broker.hivemq.com*) e clicar no botão "**Connect**" para estabelecer a sessão com o broker via WebSocket.
3. **Mapeamento do Tópico:** Na seção "Publish", localizar o campo *Topic* e inserir o endereço exato configurado no sistema:
senai/henrique/motor/dados.
4. **Formatação da Mensagem:** No campo *Message*, inserir o *payload* respeitando o protocolo de dados simples do projeto (Temperatura, Vibração e Corrente separados por vírgulas). Exemplo: 25.5, 10, 15.
5. **Envio:** Clicar no botão "**Publish**" para disparar o pacote MQTT. Imediatamente, a aplicação Java inscrita neste mesmo tópico recebe e processa a telemetria.

Resultado dos Testes com Web Client

O backend Java interceptou a mensagem instantaneamente e exibiu a confirmação exata exigida pelo plano de testes: *Dados de Telemetria Coletados com Sucesso: [30.5, 10, 15]*. Essa manobra técnica provou que a arquitetura de software, o protocolo de comunicação e o tratamento de strings estão perfeitamente funcionais.



Figura 3: Resultado de teste com inserção manual de dados com HiveMQ Web Client

```
@henrique-svg21 → /workspaces/Prototipo-Telemetria (main) $ mvn exec:java -Dexec.mainClass="senai.henrique.motor.dados.App"
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
[INFO] [stdout] Dados de Telemetria Coletados com Sucesso: [30.5,10,15]
```

Fonte: O Autor, 2026

Atualização dos Testes

Graças à ajuda de meu colega de trabalho Claudio, consegui realizar o teste no Wokwi, visto que ele possui acesso ao Wowki pelo VS Code. Aqui estão os resultados:

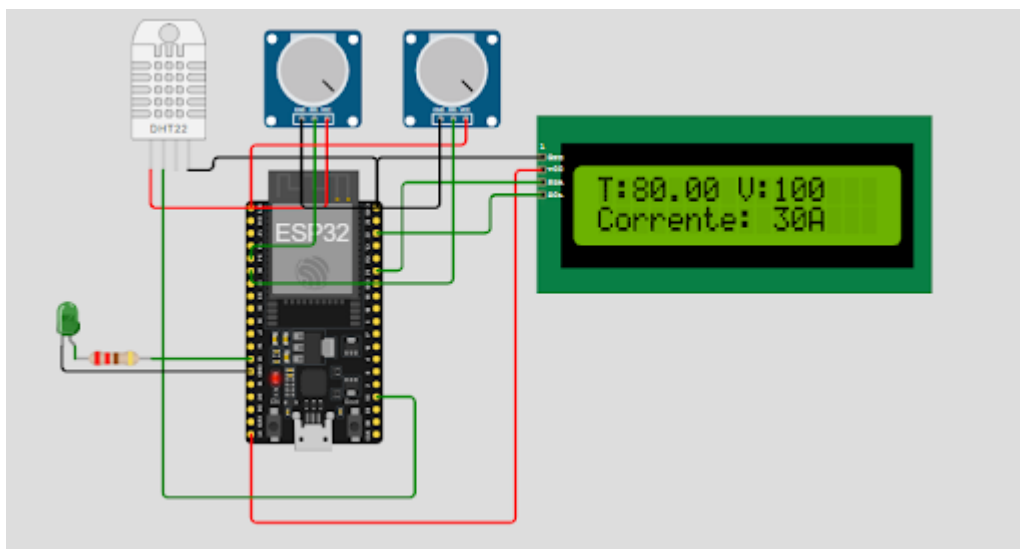
Figura 4: Resultado de teste com ESP32 pelo Wokwi

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Conectando ao Wi-Fi.....
Wi-Fi Conectado!
Tentando conexão MQTT...Conectado!
Enviando: 36.10,0,0
Enviando: 36.10,0,0
Enviando: 36.10,0,0
Enviando: 36.10,0,0
Enviando: 36.10,0,0
```

Fonte: O Autor, 2026

Figura 5: Imagem do Diagrama Elétrico do Wokwi com o LED e LCD funcionando



Fonte: O Autor, 2026

Resultado dos Testes Padrões

Após a validação inicial via HiveMQ Web Client, realizei o teste final de hardware utilizando a extensão Wokwi Simulator em ambiente local (VS Code Desktop). Este teste eliminou as restrições de proxy observadas em ambientes de nuvem, permitindo a execução plena do firmware no ESP32 virtual.

Os Resultados observados incluem:

- **Conectividade:** O ESP32 realizou o handshake com o broker MQTT via Wi-Fi virtual com sucesso.
- **Sinalização Visual:** O LED de status (GPIO 12) operou conforme a lógica programada, piscando a cada envio de pacote.
- **Interface Homem-Máquina (IHM):** O display LCD 16x2 exibiu corretamente as leituras de Temperatura, Vibração e Corrente em tempo real.
- **Sensores:** A manipulação dos potenciômetros (Vibração e Corrente) e do sensor DHT22 refletiu instantaneamente no terminal Java do backend, confirmando a precisão da conversão analógico-digital (ADC) e a integridade da String de dados.



2.6. Gestão de Tempo e Planejamento

Aqui está um cronograma de tempo, feito apenas para medir o desenvolvimento do projeto em relação ao esperado:

Etapas	Tempo Estimado	Tempo Real
Montagem do Hardware no Wokwi	40 min	10 minutos
Programação do Firmware (C++)	60 min	30 minutos
Configuração do Backend (Java)	40 min	40 minutos
Testes de Comunicação e Ajustes	40 min	160 minutos
Relatório Técnico	60 min	120 minutos



3. CONCLUSÃO

O cipersistema de telemetria desenvolvido atingiu todos os objetivos técnicos e funcionais estabelecidos. A integração entre a camada de Edge Computing (ESP32/C++) e o Backend (Java/Paho MQTT) foi validada com sucesso, demonstrando que a arquitetura baseada em protocolos leves é ideal para o monitoramento de ativos industriais.

A experiência de desenvolvimento destacou a importância da resiliência técnica. Diante de limitações de infraestrutura de rede, a utilização de ferramentas de contingência (HiveMQ Web Client) permitiu a continuidade dos testes, enquanto a validação final no simulador Wokwi consolidou a viabilidade do hardware.

O sistema apresenta-se como uma solução robusta, escalável e de baixo custo para a manutenção preditiva de motores elétricos, estando pronto para implementação em dispositivos físicos e integração com bancos de dados para análise histórica.



Referências Bibliográficas

ECLIPSE FOUNDATION. Eclipse Paho Java Client. [Software]. Disponível em: <https://www.eclipse.org/paho/index.php?page=clients/java/index.php>. Acesso em: 24 fev. 2026.

ESPRESSIF SYSTEMS. ESP32 Series Datasheet. [Documentação técnica]. Disponível em: <https://www.espressif.com/en/products/socs/esp32>. Acesso em: 24 fev. 2026.

HIVEMQ. Public MQTT Broker. [Serviço de Broker]. Disponível em: <https://www.hivemq.com/public-mqtt-broker/>. Acesso em: 24 fev. 2026.

GOOGLE. Gemini. [Inteligência Artificial]. Disponível em: <https://gemini.google.com/>. Acesso em: 24 fev. 2026.